

Z Devops: Navigating your route to live with Git

Modernizing testing, promotion and deployment

Ian J Mitchell

Dennis Behm

Table of contents

0.1	Agenda	2
0.2	Setting the context	2
0.3	The Guide	3
0.4	Today's topic - Branching Strategy for Mainframe Teams	3
0.5	... don't just take our word for it	4
1	The Branching Strategy (Recap)	4
1.1	Starting Simple	4
1.2	Starting Simple	5
1.3	Preparing to Merge into <code>main</code>	5
2	Environment branches - yes, no or meh!	6
2.1	The Route to Live	6
2.2	Branches for environments	6
2.3	Hanging on to legacy workflows	6
2.4	Starting a development cycle	7
2.5	Completing a feature	7
2.6	And another one...	8
2.7	Testing features together	9
2.8	Testing and fixing problems	10
2.9	Deploying to production	10
2.10	Fixing production problems	11
2.11	The next development cycle	12
3	Comparing to the recommended model	15
3.1	Names, words and interpretation	15
3.2	Changing the vocabulary	15

4	It's all the same!	17
4.1	Starting a development cycle	17
4.2	Features...	18
4.3	Integrating features	19
4.4	Testing and fixing problems prior to production deployment	21
4.5	Deploying to production	22
4.6	Fixing production problems	23
4.7	The next development cycle	25
4.8	Advantages of the recommended model	27
4.9	<i>DEV</i> is a series of Epics	27
4.10	Equivalence	27
4.11	<i>TEST</i> is <i>main</i>	27
4.12	<i>PROD</i> is a series of Releases	28
4.13	What else is different?	28
5	Summary	29
5.1	Summary	29
6	THANK YOU!	29

0.1 Agenda

- Recap the basics of the recommended branching scheme
 - Focuses on the developer workflow
- “Environment branches” - yes, no or meh!
 - What’s the point in hanging on to old practices?
 - It’s just words... but words matter!
- The pipelines in action - demo time!

0.2 Setting the context

- Devops is foundational to modernization.
- Modernization of Devops is no longer a voyage of exploration.
 - (But plenty people will still try to convince you otherwise!)
- No single tool or approach covers the entire journey to the destination.

We have a [Guide](#)... yes, it would be better if it were a map!

0.3 The Guide

IBM Z® DevOps Acceleration Program Evaluate Learn Plan Transform Use Iterate

IBM Z DevOps Guide
Enabling the enterprise DevOps transformation journey

IBM® strongly believes that DevOps transformations are anything but plug-and-play - with this Guide we map your journey with a tried and tested approach appealing to all stakeholders.

- Evaluate**
Understand the benefits of enterprise DevOps, what's involved, and the approaches to take.
 - Applying DevOps to IBM Z
 - DevOps transformation milestones
- Learn**
Get familiar with the DevOps tools, roles, and culture.
 - CI/CD for z/OS
 - Git basics
- Plan**
Design the strategy for your enterprise DevOps transformation.
 - Planning repositories
 - Managing dependencies
- Transform**
Implement your new developer experience, CI/CD pipeline, and DevOps practices.
 - Setting up CI/CD tools
 - Migrating data to Git
 - Implementing pipeline actions
- Use**
Work effectively to deliver value with the new tools and practices.
 - Git-based working practices
 - Audits and compliance
- Iterate**
Scale and enhance your DevOps toolchain.
 - Next steps after a POC/pilot (sample doc)
 - Additional tools

0.4 Today's topic - Branching Strategy for Mainframe Teams

IBM Z® DevOps Acceleration Program Docs News

Overview
Applying DevOps to IBM Z
CI/CD for z/OS applications
Getting started
Working practices
Introduction to working practices for teams using Git
Git branching model for mainframe development
Working practice variations
Audit and compliance
Tutorials and courses
Designing the CI/CD pipeline
Migrating data from z/OS to Git
Implementing CI/CD
Resources

Working practices > Git branching model for mainframe development

The Git branching model for mainframe development

As Git became the de facto version control system in today's IT world, new terminologies such as "repositories", "branches", and "merges" arose. By agreeing upon a central Git server to integrate and consolidate changes, development teams were able to collaborate more efficiently and effectively. Building upon the open-source vanilla Git implementation, popular Git providers including GitHub, GitLab, and Bitbucket have implemented additional workflow features to facilitate a secure and stable development process. These include features such as pull requests (sometimes referred to as "merge requests") to support coordination with Git in larger teams. The term "pull request" will be used throughout this page to designate the operation of reviewing and merging one branch into another.

Many mainframe development teams follow a release-based or iteration-based process to deliver incremental updates to a pre-defined production runtime.

On this page
Characteristics of trunk-based development with feature branches
Starting simple
Scaling up
Naming conventions
Integration branches
Workflows in this branching model
Deliver changes with the next planned release
Implement a fix for the current production state
Use an epic branch for a significant development initiative
Learn more

0.5 ... don't just take our word for it

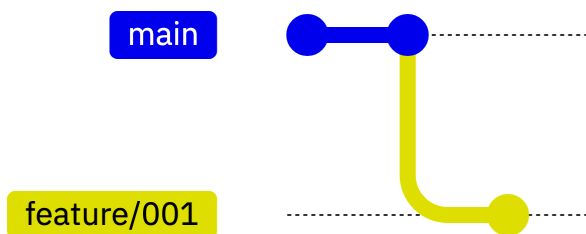
The screenshot shows the Microsoft Azure DevOps Services documentation page. The header includes the 'Learn' section with navigation links like 'Discover', 'Product documentation', 'Development languages', and 'Topics'. The main navigation bar lists 'Azure DevOps Services', 'Build and Deploy Apps', 'Get Started', 'Tools and settings', and 'Resources'. The article title is 'Adopt a Git branching strategy', dated 03/25/2024, with 12 contributors. The left sidebar shows a 'Version' dropdown set to 'Azure DevOps Services' and a 'Filter by title' search bar. Below this, a 'Git' section is expanded, showing 'Git Repositories' and 'Branches & forks', with 'Branch strategy' selected. The main content area has an 'In this article' section with links to 'Keep your branch strategy simple', 'Manage releases', and 'Manage deployments'. The article text begins with 'Azure DevOps Services | Azure DevOps Server 2022 - Azure DevOps Server 2019' and discusses distributed version control systems like Git. The right sidebar contains 'Additional resources' including 'Training' (Design and implement branch strategies and workflows - Training) and 'Documentation' (Branch policies - Azure Repos).

Microsoft Azure DevOps Services

1 The Branching Strategy (Recap)

1.1 Starting Simple

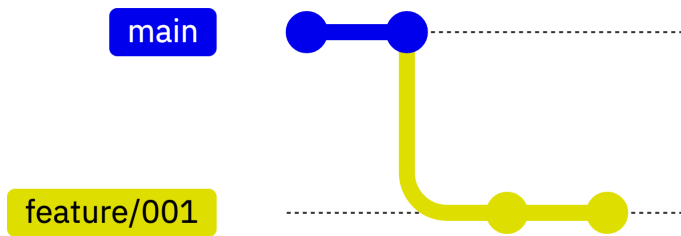
Every change starts in a branch.



- Developers work in the branch to make changes, perform user builds and unit tests.
- A branch holds multiple commits (changes to multiple files).

1.2 Starting Simple

Every change starts in a branch.



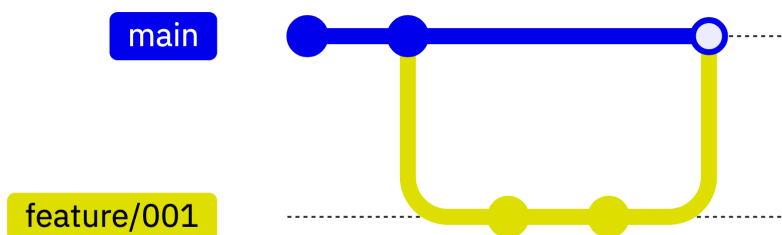
These changes on these branches are

- built,
- tested,
- reviewed and
- approved before merging to `main`.

1.3 Preparing to Merge into `main`

Feature Team/Developers will:

- Build
 - Builds may be done to any commit on any branch
 - Feature branch **must** build cleanly for a Pull Request
- Test
 - To prove quality of the changes in their feature branch



2 Environment branches - yes, no or meh!

2.1 The Route to Live

Comparing the recommended branching scheme to some common legacy patterns for workflows and build/deploy schemes

- These environments dominate the terminology and workflows with which the team are familiar
- So there is an adjustment required to be fully comfortable with the recommended model

2.2 Branches for environments

Typically

- *DEV*
- *TEST*
- *PROD*

Let's look at the workflow for a set of features in a release.

2.3 Hanging on to legacy workflows

The following diagrams adopt the old names for:

- branches where developers work (*'DEV'*),
- branches representing versions of the code base which are built and deployed to systems where formal tests are executed (*'TEST'*) and
- branches representing the state of the deployed system in production (*'PROD'*)

and shows associated build pipelines and systems on to which the built packages are deployed.

The pipelines targeting each environment may inject some characteristics specific to each environment.

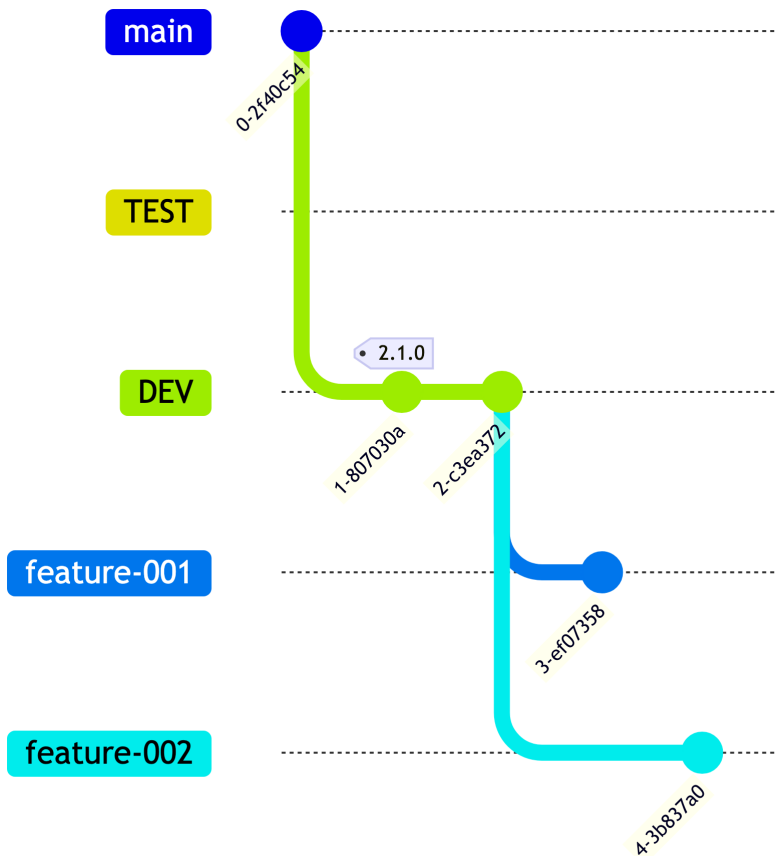
Obviously we maintain the principle that features will be implemented first in *DEV* and progress through one or more *TEST* branches and environments before deployment to *PROD*.

Equally obviously, *feature* branches are used by developers before they merge into *DEV*.

In order to be included in the pipeline build for any environment, the commits implementing a feature must be merged into the branch for that environment. Build°vs are performed for any environment.

When one or more features are code-complete and merged into the *DEV* branch a new build is performed for deployment to the *DEV* environment.

2.4 Starting a development cycle

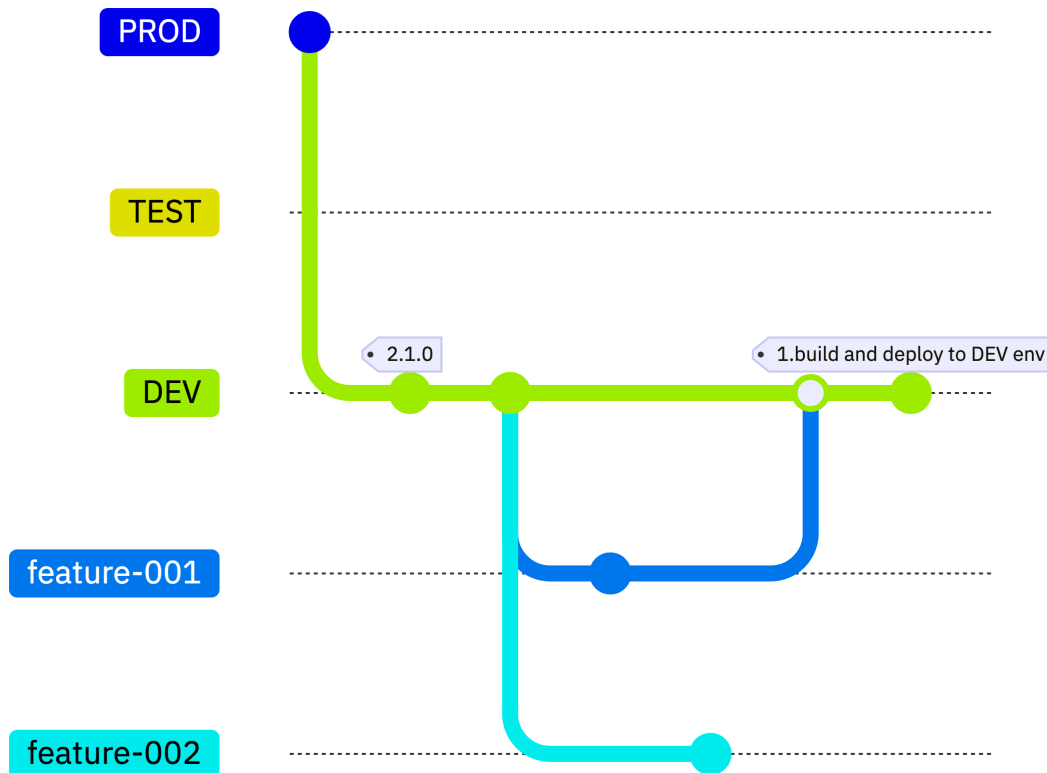


Both features for *release 2.1.0* are developed in their own dedicated branches.

2.5 Completing a feature

Feature-001 completes first and a PR is used to review and approve its merge into the *DEV* branch.

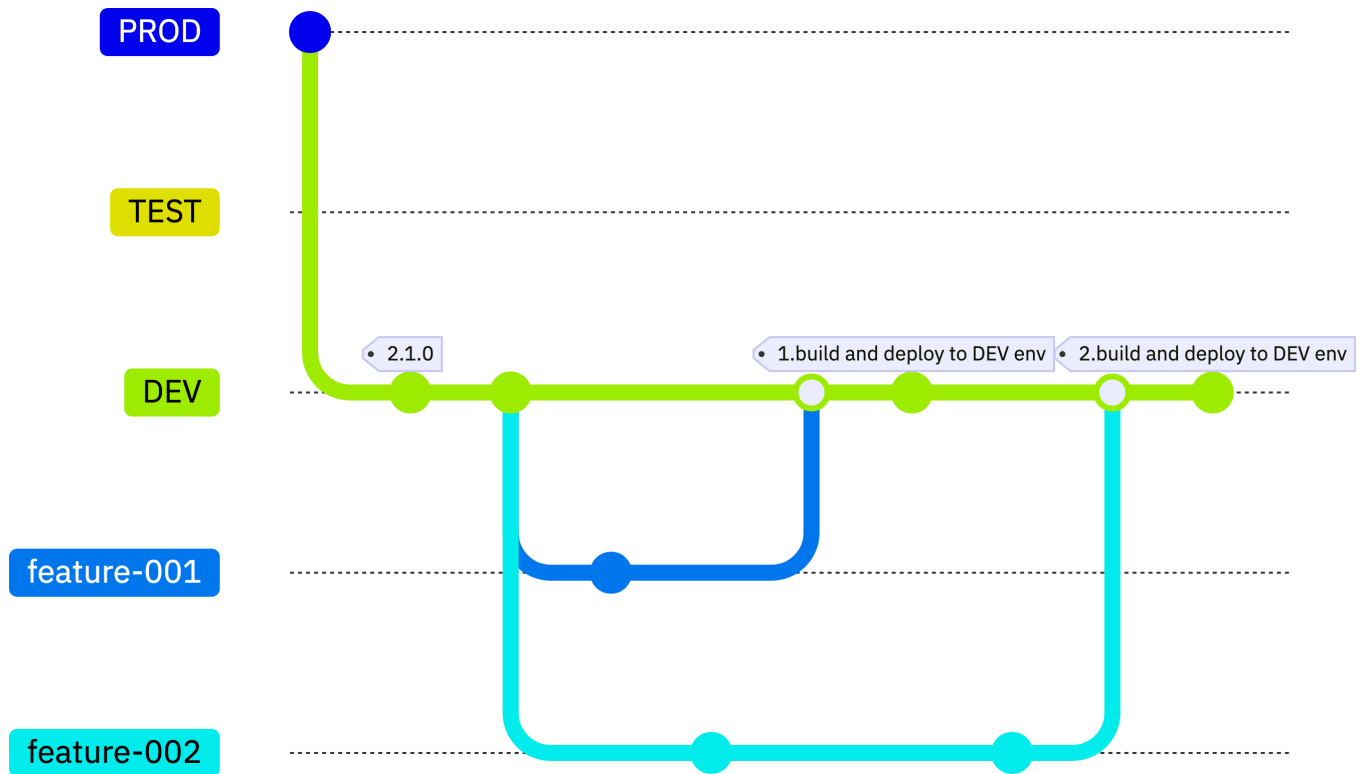
A new build of *DEV* is deployed to a *DEV* **environment**.



2.6 And another one...

The same workflow is performed when the second feature planned for the release is ready to be merged into *DEV*.

It's implicit that the feature-002 branch is synchronized with *DEV* after feature-001 is merged to *DEV* (at least before feature-002 can be merged with *DEV*).

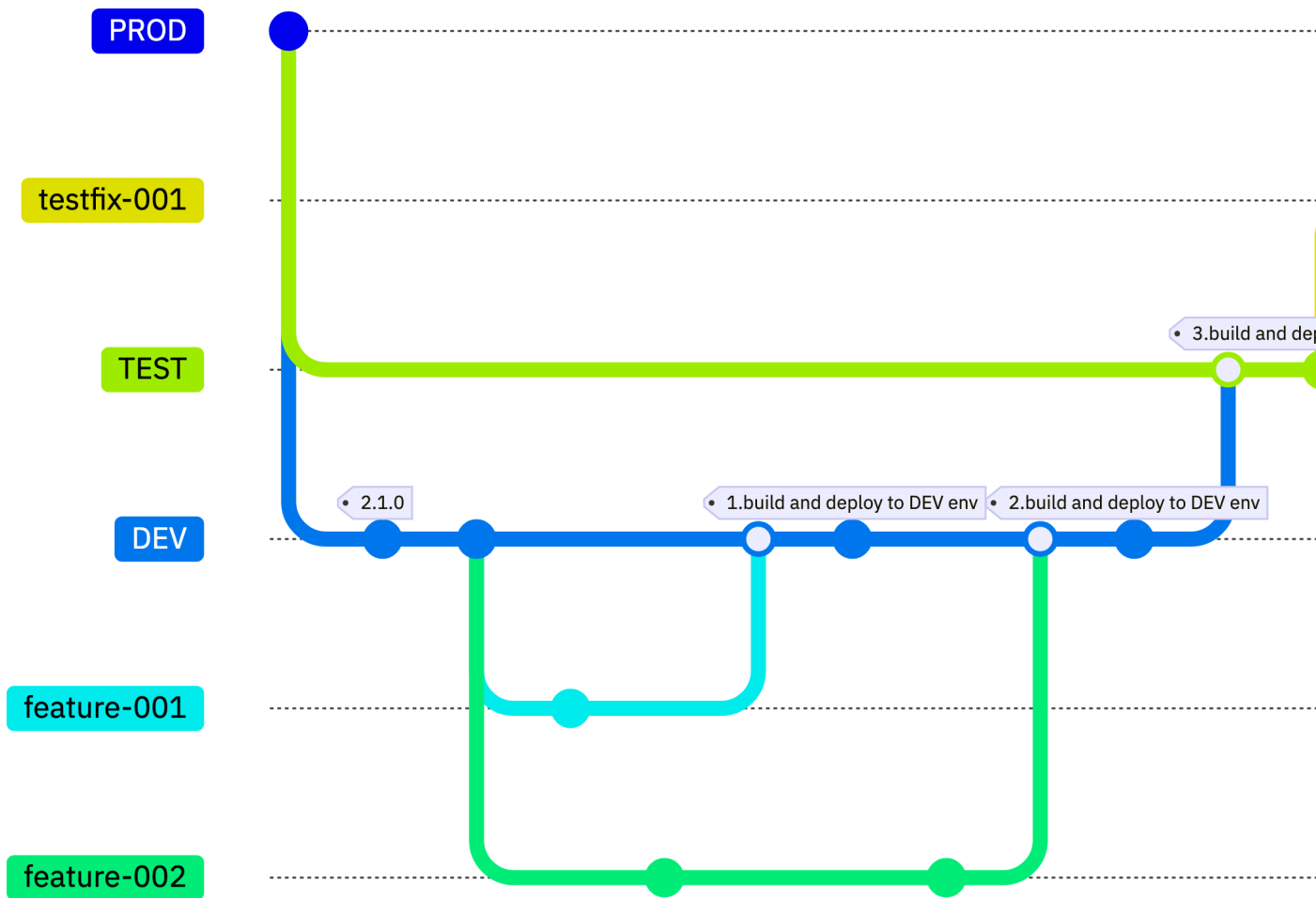


2.7 Testing features together

As these two features are now merged onto the *DEV* branch, they can be promoted together onto the *TEST* branch

- By performing a new combined build and deployment to the *TEST environment*.
- Here a problem is discovered and a fix to *TEST* is developed in a branch, merged into *TEST*, built and deployed for validation.

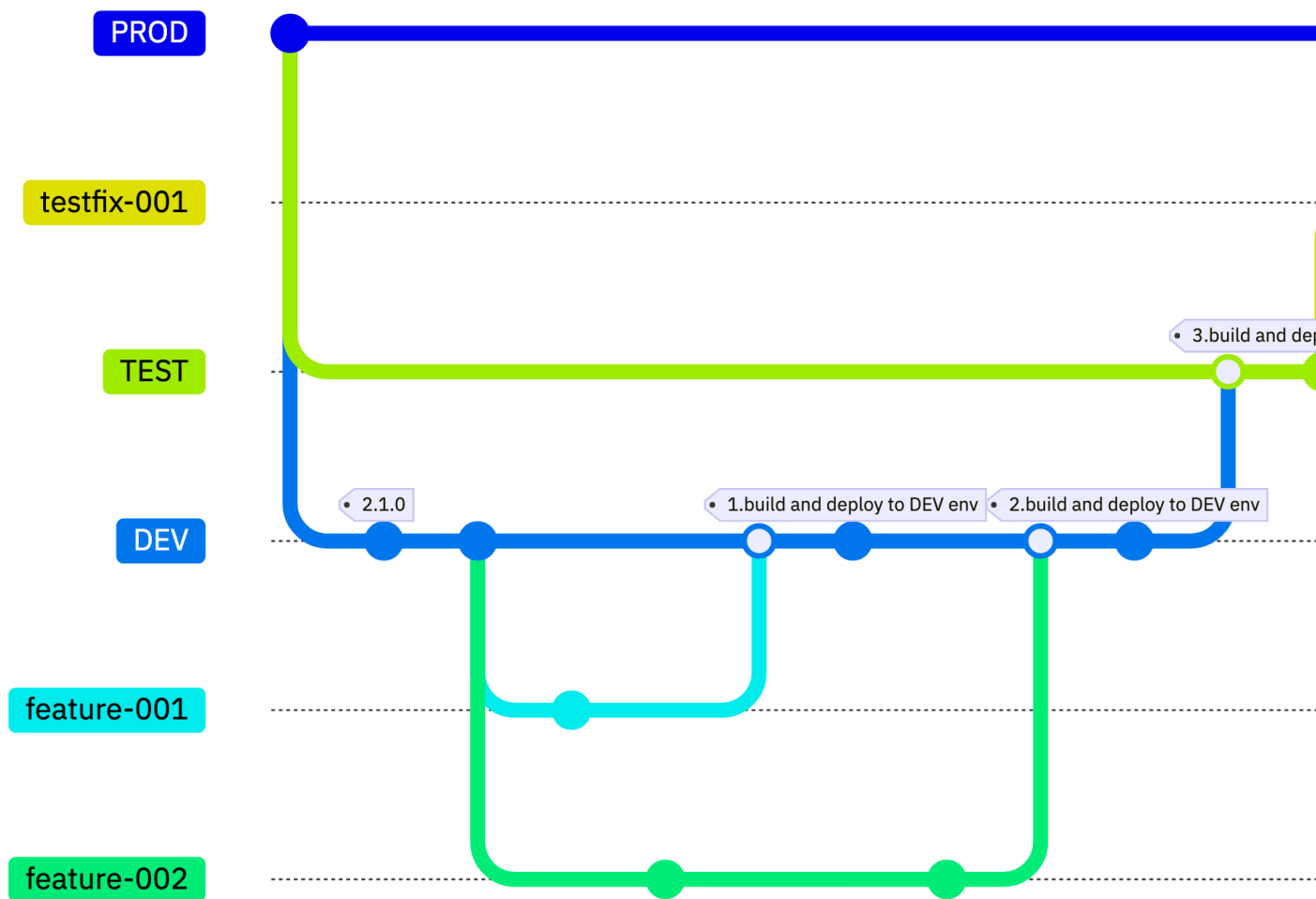
2.8 Testing and fixing problems



2.9 Deploying to production

After a successful test cycle, a PR is used to review and approve the merging of *TEST* (which now includes both planned new features) into the *PROD* branch.

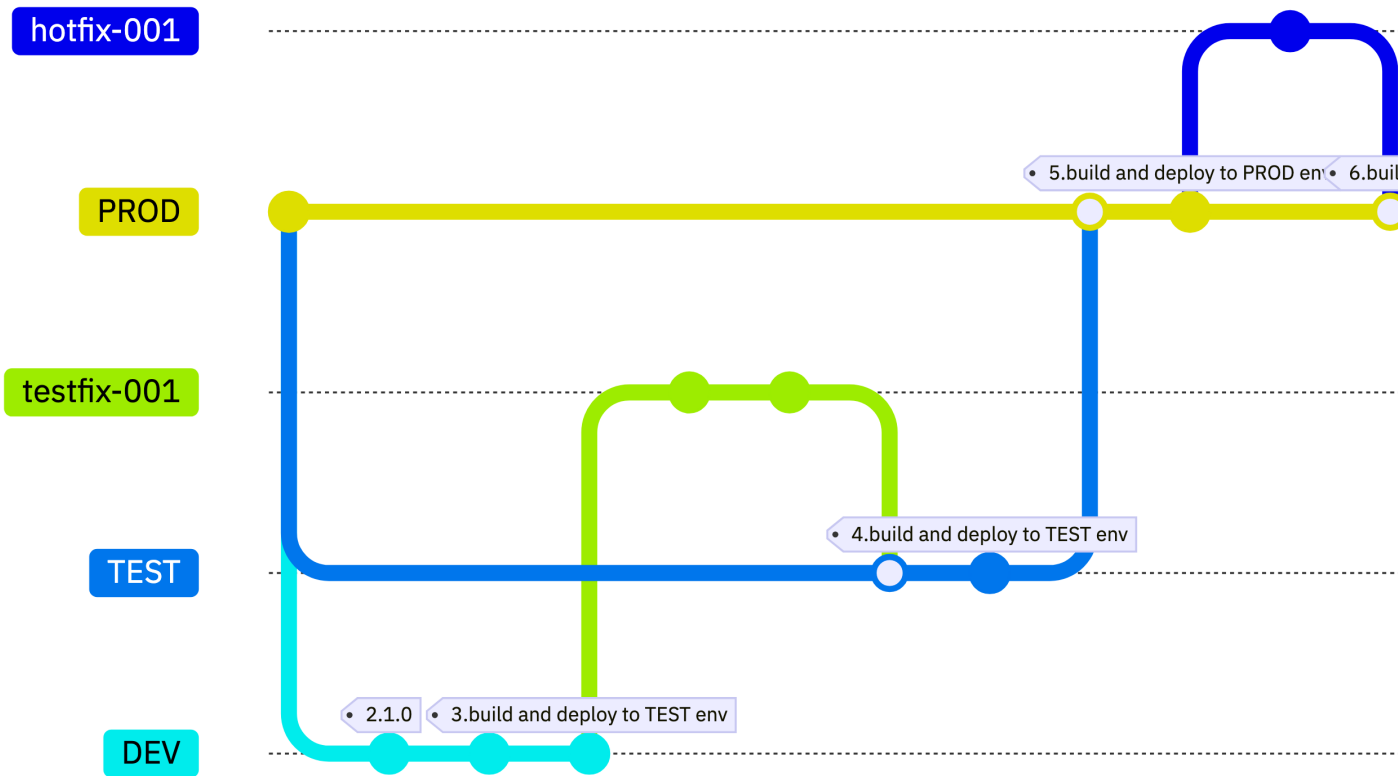
This is followed by a build and deployment to the *PROD environment*.



2.10 Fixing production problems

A HotFix is needed.

This is developed, reviewed and merged into the *PROD* branch with a new build and deployment to the *PROD environment* performed.

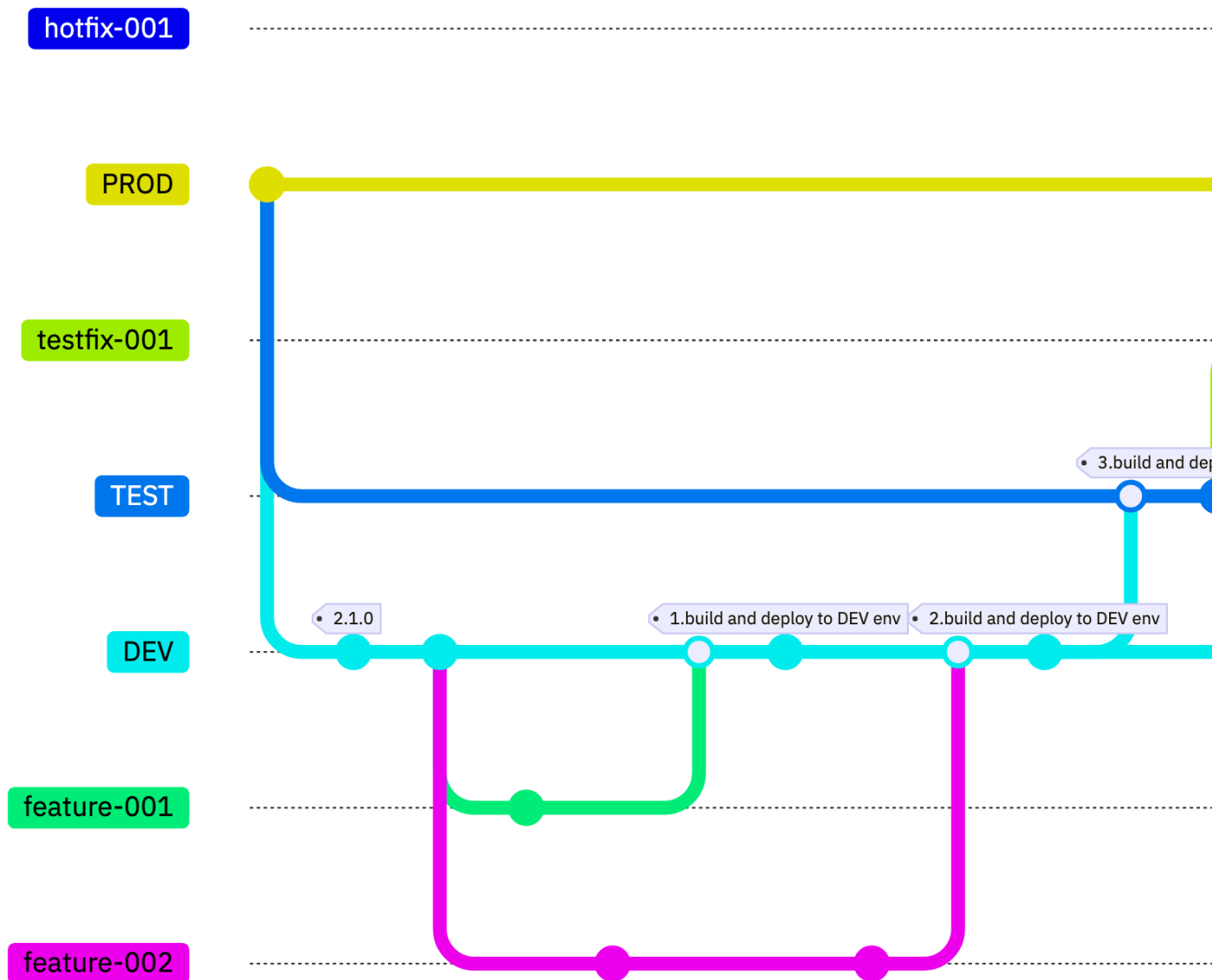


2.11 The next development cycle

Now the testing and release cycle for *release 2.1.0* is complete, both the *TEST* and *PROD* branches have fixes which need to be synchronized to *DEV*.

Since all the commits missing from *DEV* are present on *PROD*, we just `git pull` from *PROD* to *DEV*.

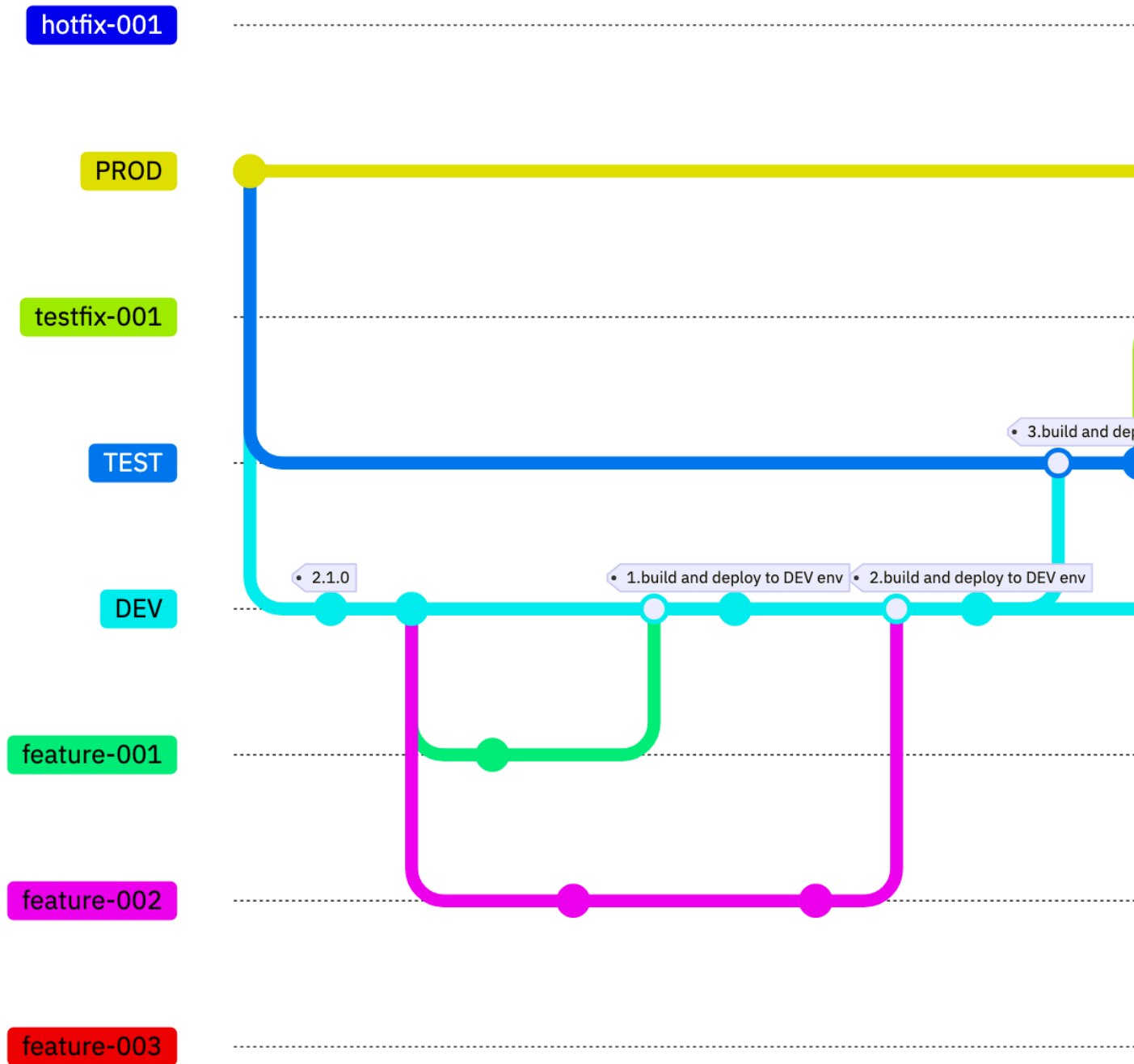
2.12



DEV is now identical to *PROD*.

Just as at the beginning of *release 2.1.0*, a tag is used to demarcate the beginning of a new release cycle.

2.13



The cycle for *Release 2.2.0* goes smoothly with Feature-003 reviewed, approved, merged and tested into *DEV*, *TEST* and *PROD* with no need for any fixes!

(Which means there are no commits made to either *TEST* or *PROD* that need to go back to *DEV*.)

3 Comparing to the recommended model

3.1 Names, words and interpretation

The recommended model uses the industry-standard terms for branches in workflows reflecting common views of the roles they play.

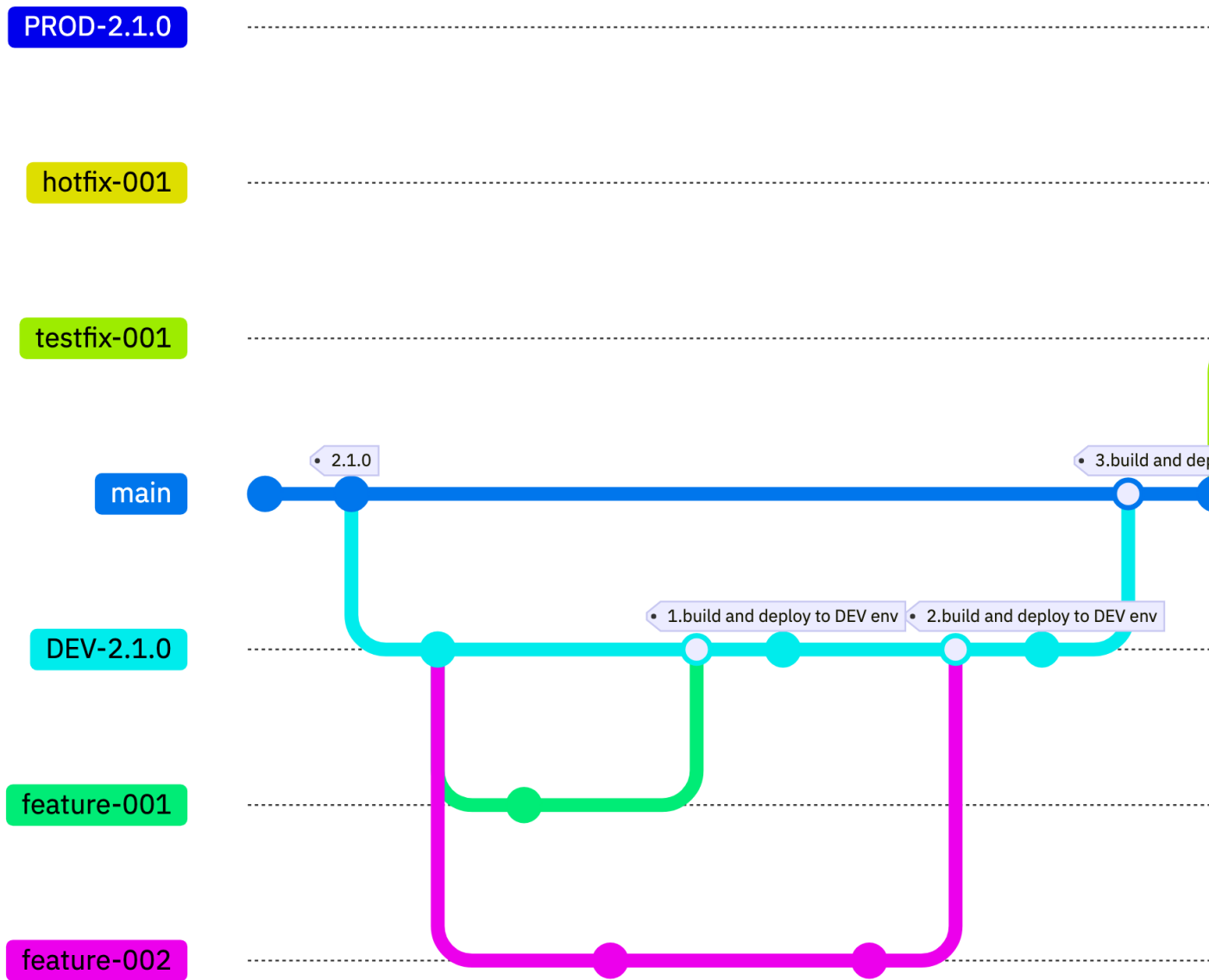
- Do these conflict with the legacy terms, or are they interpretations of the model achieving equivalent outcomes?
- Is the legacy interpretation fundamentally different or close enough not to be a blocker on adoption?

3.2 Changing the vocabulary

The following diagram shows the same basic workflow actions and elements, but with the names from the recommended model.

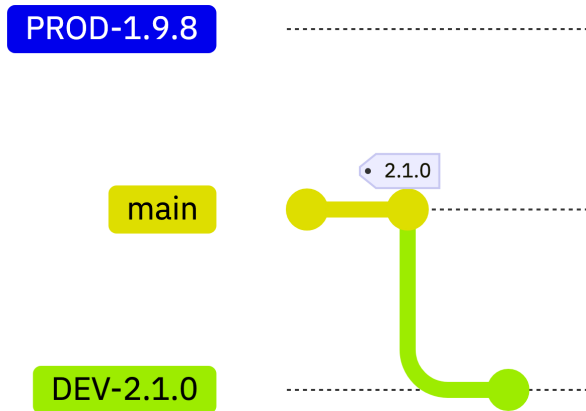
- *DEV* is equivalent to a series of Epics
- *TEST* is equivalent to `main`
- *PROD* is equivalent to a series of Releases

3.3



4 It's all the same!

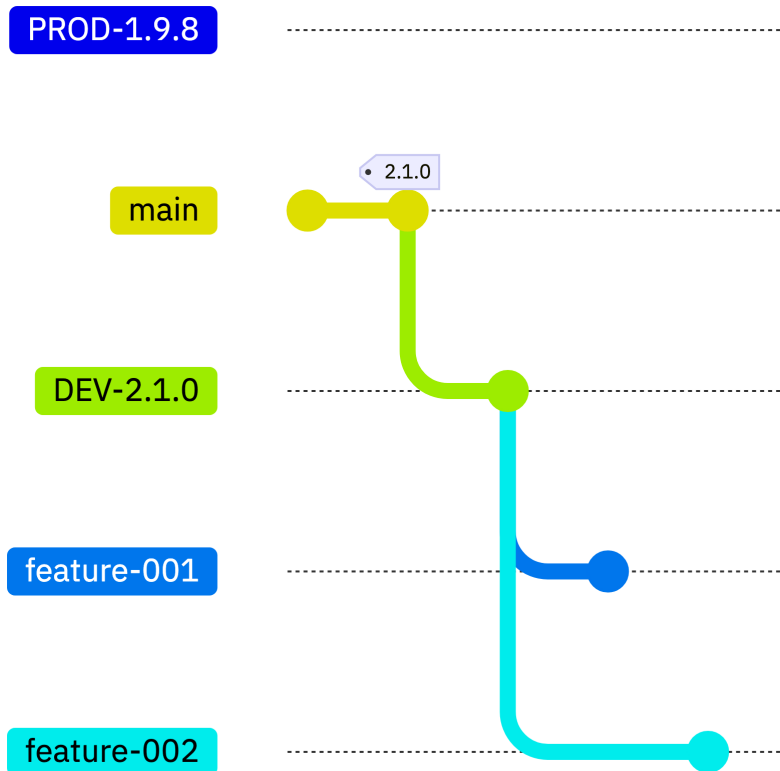
4.1 Starting a development cycle



`main` is the most up to date branch at the beginning of every cycle. Release 1.9.8 is currently deployed to production via a previous cycle which built, tested and deployed it from a dedicated branch.

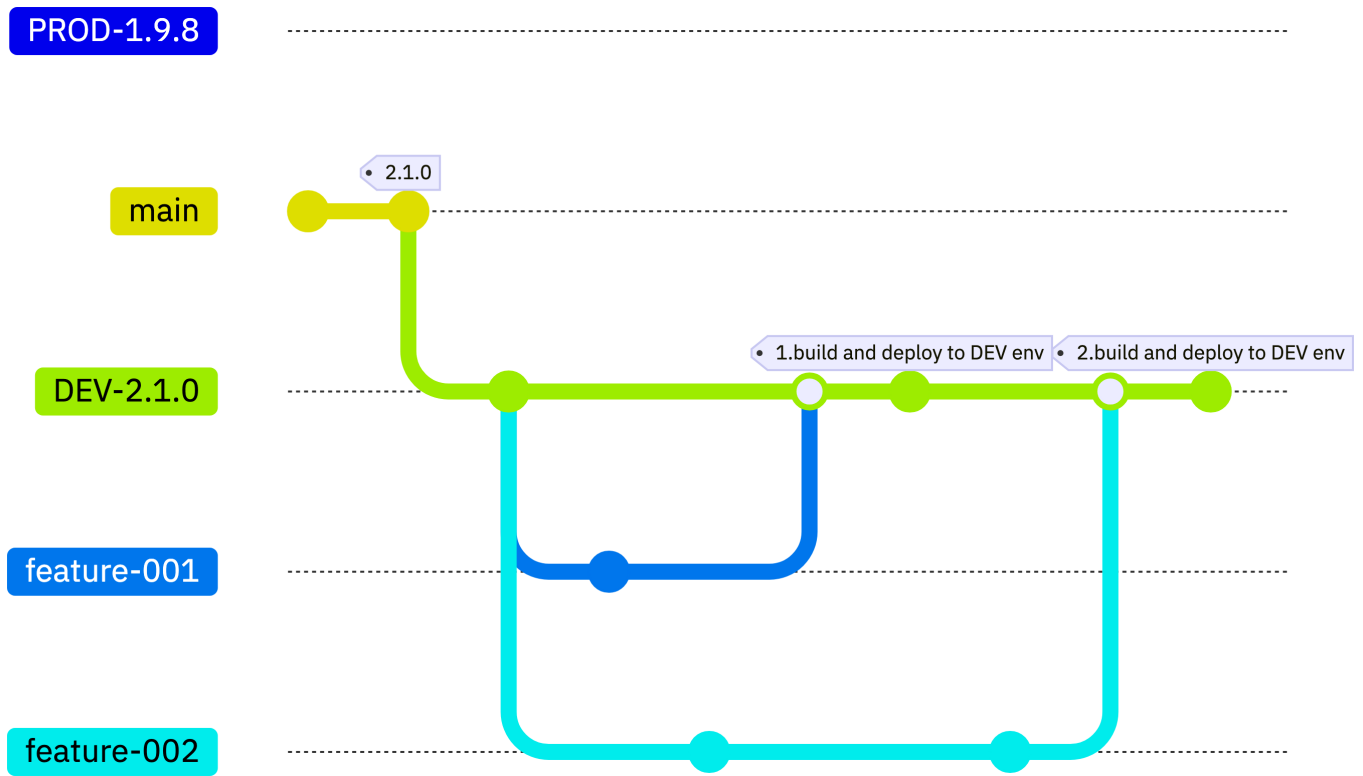
A new `DEV-2.1.0` branch is created as an epic branch to be the integration branch for the features which will go into this next release cycle.

4.2 Features...



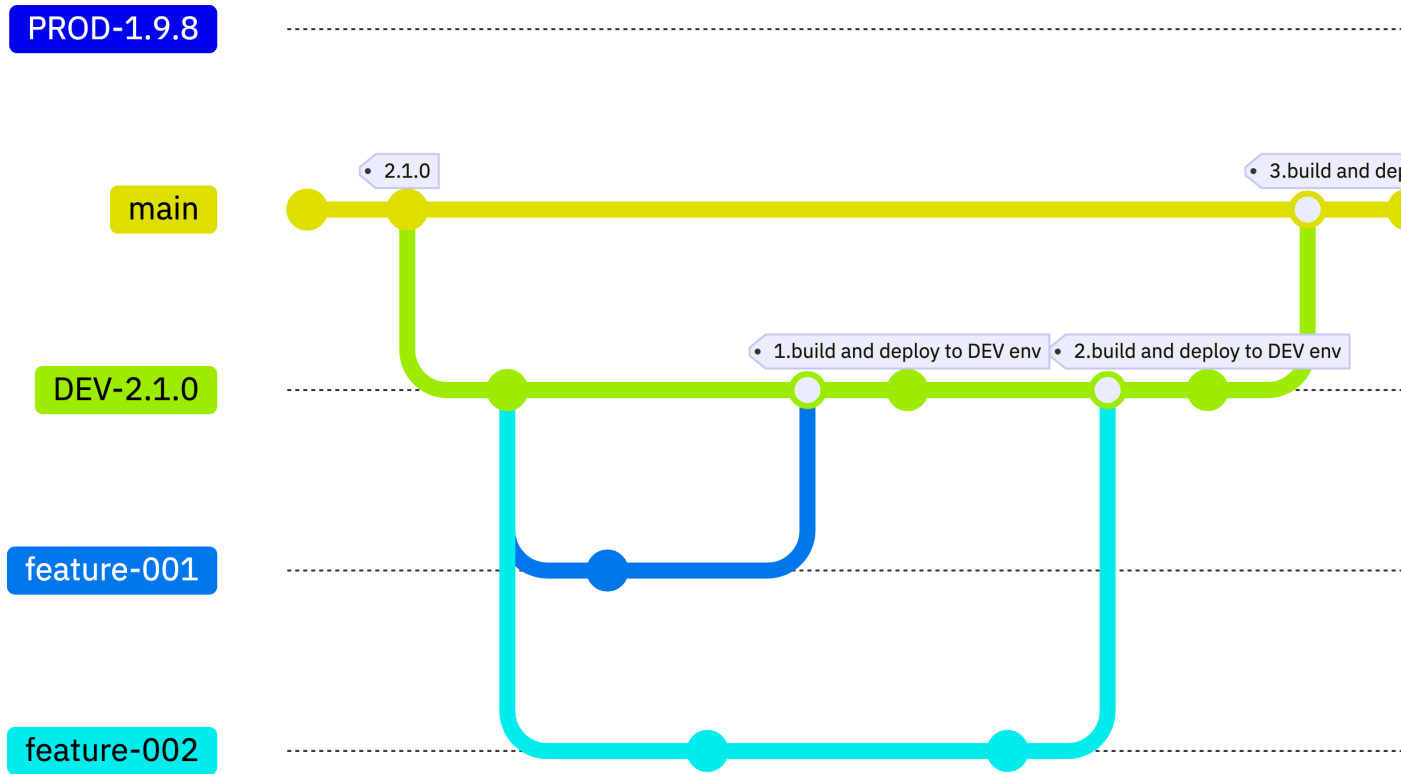
As usual, features are developed in their own branches.

4.3 Integrating features



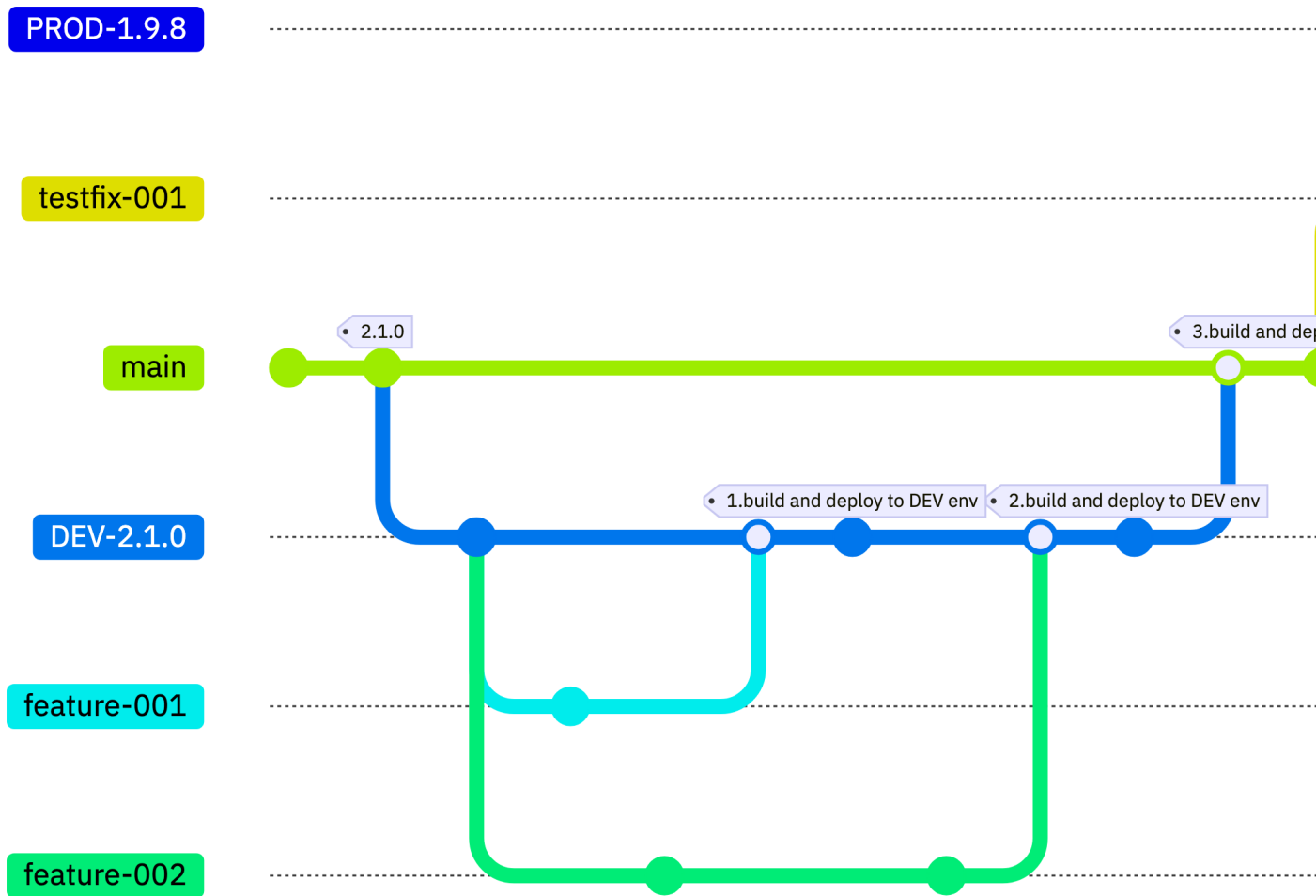
As each feature is completed, PRs are used to review, approve and merge them into the DEV-2.1.0 branch where they can be tested.

Again, it's implicit that the feature-002 branch is synchronized with DEV after feature-001 is merged to DEV (at least before feature-002 can be merged with DEV).



When both pass the tests for a DEV branch, a PR proposes a merge of them both to `main` where they can be reviewed and approved to be a release candidate.

4.4 Testing and fixing problems prior to production deployment

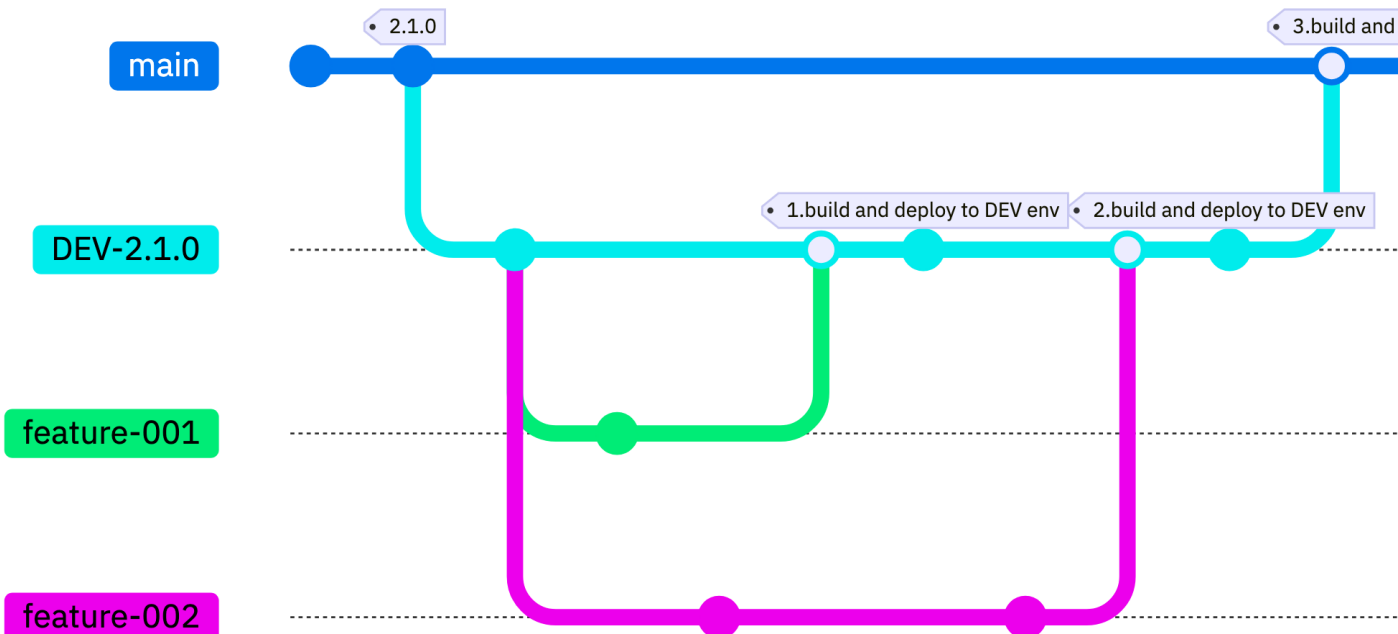


Again, the bug is discovered in testing, and `testfix-001` is required. It's developed in a branch, reviewed and approved and the build and testing refreshed.

PROD-1.9.8

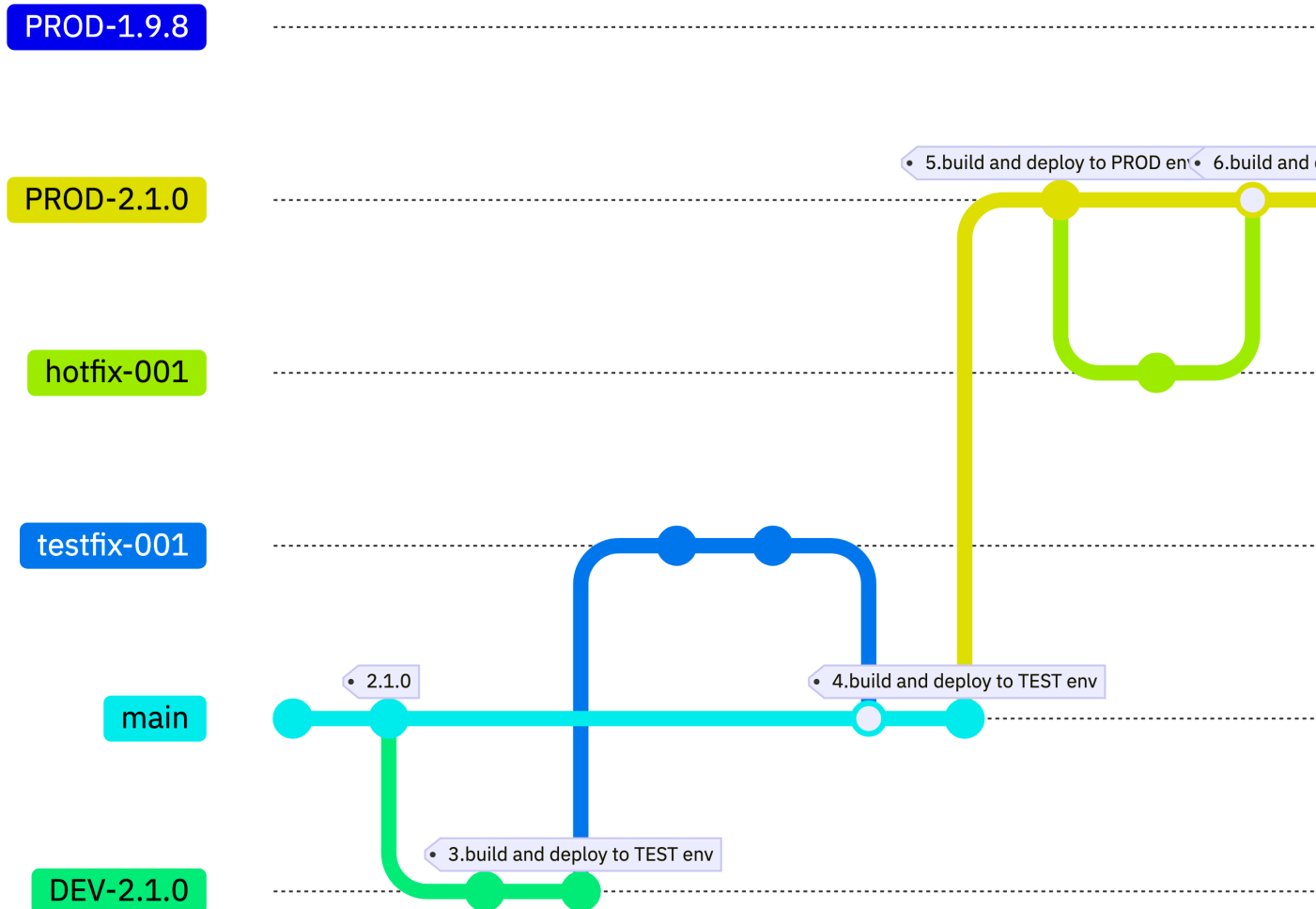
PROD-2.1.0

testfix-001



4.5 Deploying to production

Now `main` has the new features and is passing the tests, it's ready to consider a deployment to Production.



4.6 Fixing production problems

The bug which needs fixing with `hotfix-001` is again found and resolved using a branch off `PROD-2.1.0`.

PROD-2.1.0

hotfix-001

testfix-001

main

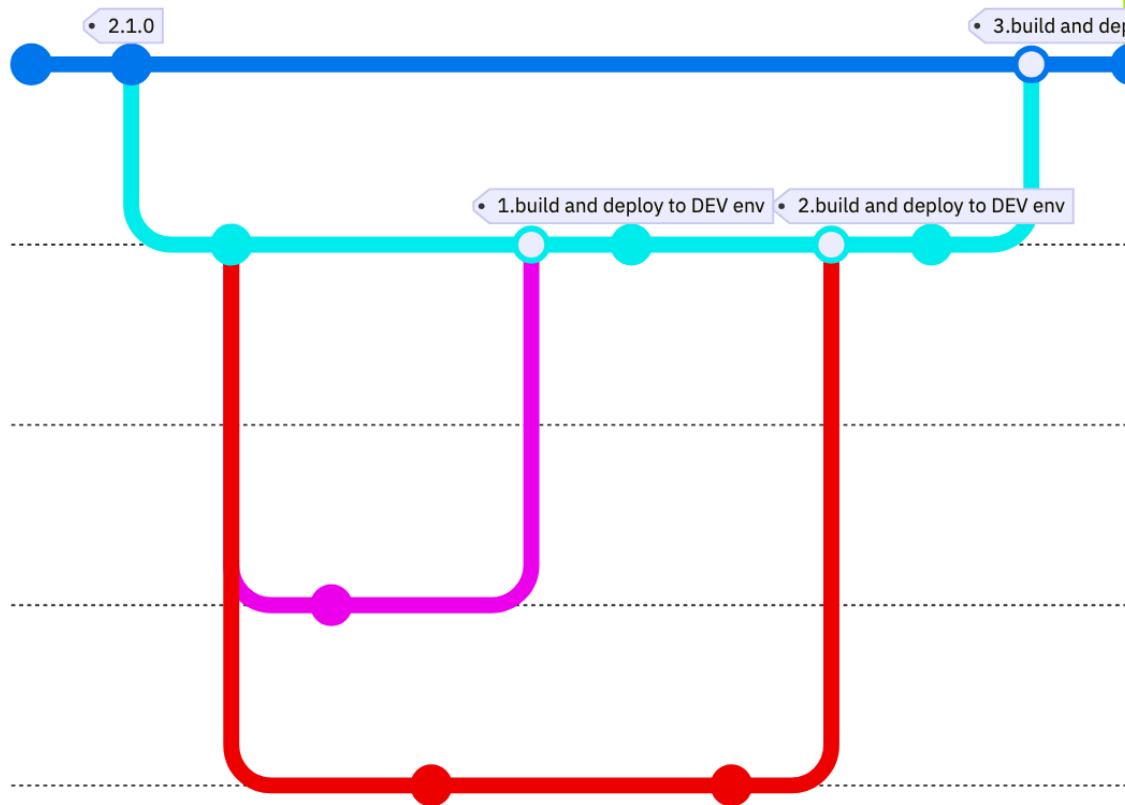
DEV-2.1.0

DEV-2.2.0

feature-001

feature-002

feature-003



4.7 The next development cycle

To continue development for a *release 2.2.0*, a new *DEV-x.y.z* branch is created. Development of features on *DEV-2.2.0* proceeds in the usual way.

The commit(s) for *hotfix-001* are merged back to `main` and thus will be present when any further commits are merged in either `main` or the *DEV-2.2.0* branches.

PROD-2.1.0

hotfix-001

PROD-2.2.0

testfix-001

main

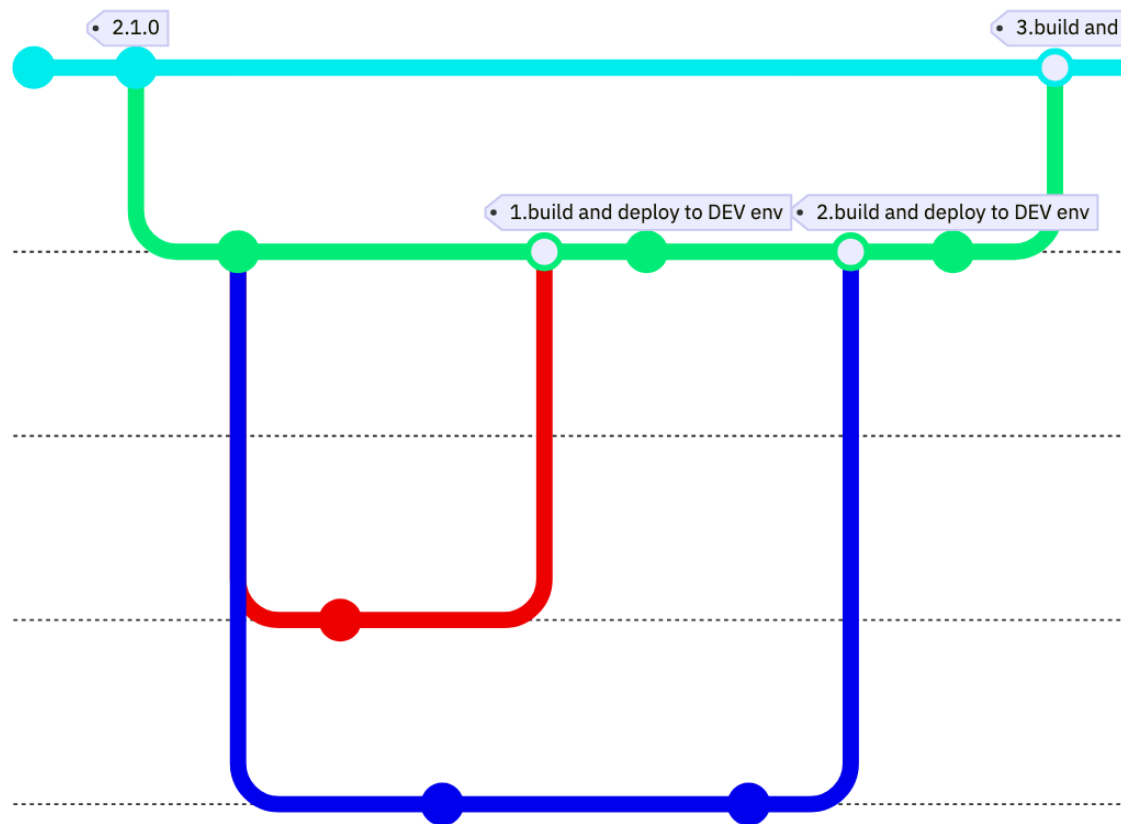
DEV-2.1.0

DEV-2.2.0

feature-001

feature-002

feature-003



Build, testing and production deployment for Release 2.2.0 then proceeds as normal.

4.8 Advantages of the recommended model

4.9 *DEV* is a series of *Epics*

The *DEV* branch integrates multiple features before they are merged into *TEST* - which is the same role that epic branches have with respect to `main` in the recommended model.

It signals that the target integration branch for developers is functionally equivalent to a series of *Epic* branches.

4.10 Equivalence

Since *DEV* would both be the target of merging new features and have to be refreshed from *TEST*, it makes **no functional difference** whether such a branch is:

- **long-lived** and rebased on the `HEAD` of *TEST* if *TEST* has fixes merged on to it OR
- created as an **Epic branch** from `HEAD` of *TEST* at the beginning of each development cycle.

One or the other of those actions **MUST** be done in order for the developers' branch to be up-to-date.

It makes NO DIFFERENCE to the history or the validity of the branch developers work off of if there's a single long-lived branch synchronised to the `HEAD` of *TEST* (aka `main`) or if such a branch is created for each cycle.

4.11 *TEST* is `main`

TEST is the integration branch for all new features - essentially identical to the branch we call `main` in the recommended model.

- In legacy terms, *DEV* would advance ahead of *TEST* with feature branches being merged into it exactly as epic branches would be ahead of `main` in the recommended model.
 - As commits from feature branches are merged into *DEV*, the *DEV* pipeline builds and deploys testable packages to the *DEV* environment. Having proven their goodness, a PR is created to merge *DEV* into *TEST*.

If nothing else has merged into *TEST* since the last PR from *DEV* then this is straightforward. But if it's permitted for developers to merge commits for fixes into *TEST* without merging them into *DEV* first, then *DEV* and *TEST* will have diverged and *DEV* will need to be brought up-to-date with the commits from *TEST* which are missing.

Commits normally merge according to the upward arrows in both diagrams, but there are a number of downward pointing arrows - these signify synchronization points where a branch which is normally a source of commits to another (*DEV* to *TEST*, *TEST* to *PROD*) need to incorporate commits which were made independently to the (normally) target branch.

This is exactly as *main* and *epic branches* are used. Thus *TEST* is playing the role we assign to *main* in the recommended model. It seems entirely reasonable that the branch to which developers aim to merge their work and which is the main stage of testing has the conventional common name of *main*.

- The *TEST* branch **IS** a long-lived (actually permanent) branch in both interpretations of the model.
 - The *TEST* branch will have tested commits for new features ready to be merged to the *PROD* branch and hence built for deployment to the *PROD* systems.

4.12 *PROD* is a series of Releases

The legacy interpretation demands that commits for new features are merged from *TEST* to *PROD*, but also that it is allowed for *hot-fixes* to be merged with *PROD* in emergencies.

- *TEST* cannot be behind *PROD* (ie missing any hot-fixes) when a new set of commits are to be released to *PROD*.
- Taking the commits from *TEST* which are newer than the last to *PROD* makes their histories identical.
- This makes it functionally the same as making a new branch off the HEAD of *TEST* (or any given point in the history of *TEST*) - this is identical to the creation of a release maintenance branch in the recommended model.

4.13 What else is different?

Having established a functional and general procedural equivalence with a scheme assuming *DEV*, *TEST* and *PROD* are just more familiar names, what else can stop the recommended model being adopted?

Any build from any commit on any branch might also use some environment-specific options injected by the pipeline - what might these options be?

- Compiler options - like compiling for debug vs performance
- where to get source and/or linkable binaries from which don't belong to the repo
- anything else?

These variations depend on the actions of the pipeline and not just the contents of the source files in the repo.

5 Summary

5.1 Summary

- Legacy terminology is a blocker for alignment to common ways of working
 - and makes on-boarding harder
- Our recommended model decouples the state of the code from the environments
 - for which the code is built
 - and where it is deployed

Words...

- *DEV* is equivalent to a series of Epics
 - oriented to projects/features
- *TEST* is equivalent to `main`
 - so let's use the standard term
- *PROD* is equivalent to a series of Releases
 - all with the history and traceability we all need

6 THANK YOU!